Aula 06 – Caminho de dados e controle MIPS Multiciclo

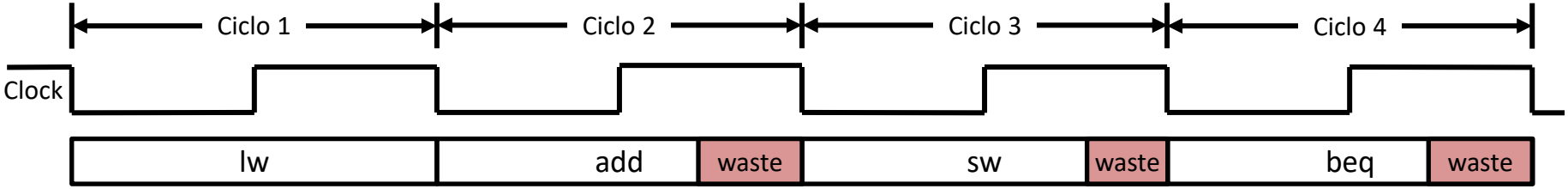
Prof. João Fernando Mari
joaofmari.github.io

Roteiro

- MIPS Monociclo
- MIPS Multiciclo – Visão de alto nível
- Executando instruções Multiciclo
- Dividindo a execução da instrução em ciclos de clock
- Definindo o controle
  - Sinais de controle de 1 bit
  - Sinais de controle de 2 bit

MIPS Monociclo

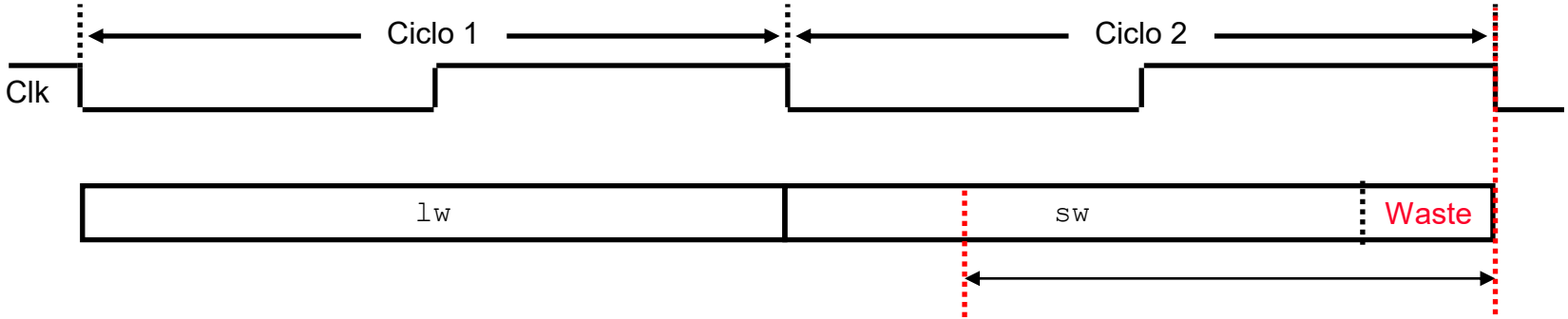
- Problema em máquinas de ciclo único



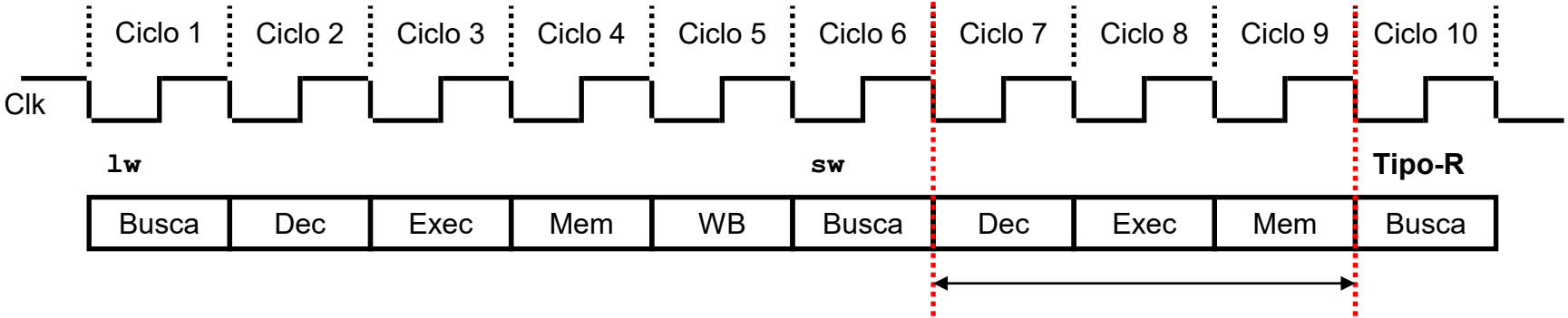
- Implementação multiciclo:
  - Cada fase de execução da instrução em um ciclo
  - Unidades funcionais podem ser compartilhadas.
    - Ou seja, podem ser usadas mais de uma vez por instrução
  - Redução da quantidade de hardware necessário
- Vantagens da implementação multiciclo:
  - Instruções são executadas em quantidades diferentes de períodos de clock
    - CPI variável.
    - Caso comum pode ser melhorado.
  - Possível compartilhar unidades funcionais
    - Redução de custo.

MIPS Monociclo Vs. Multiciclo

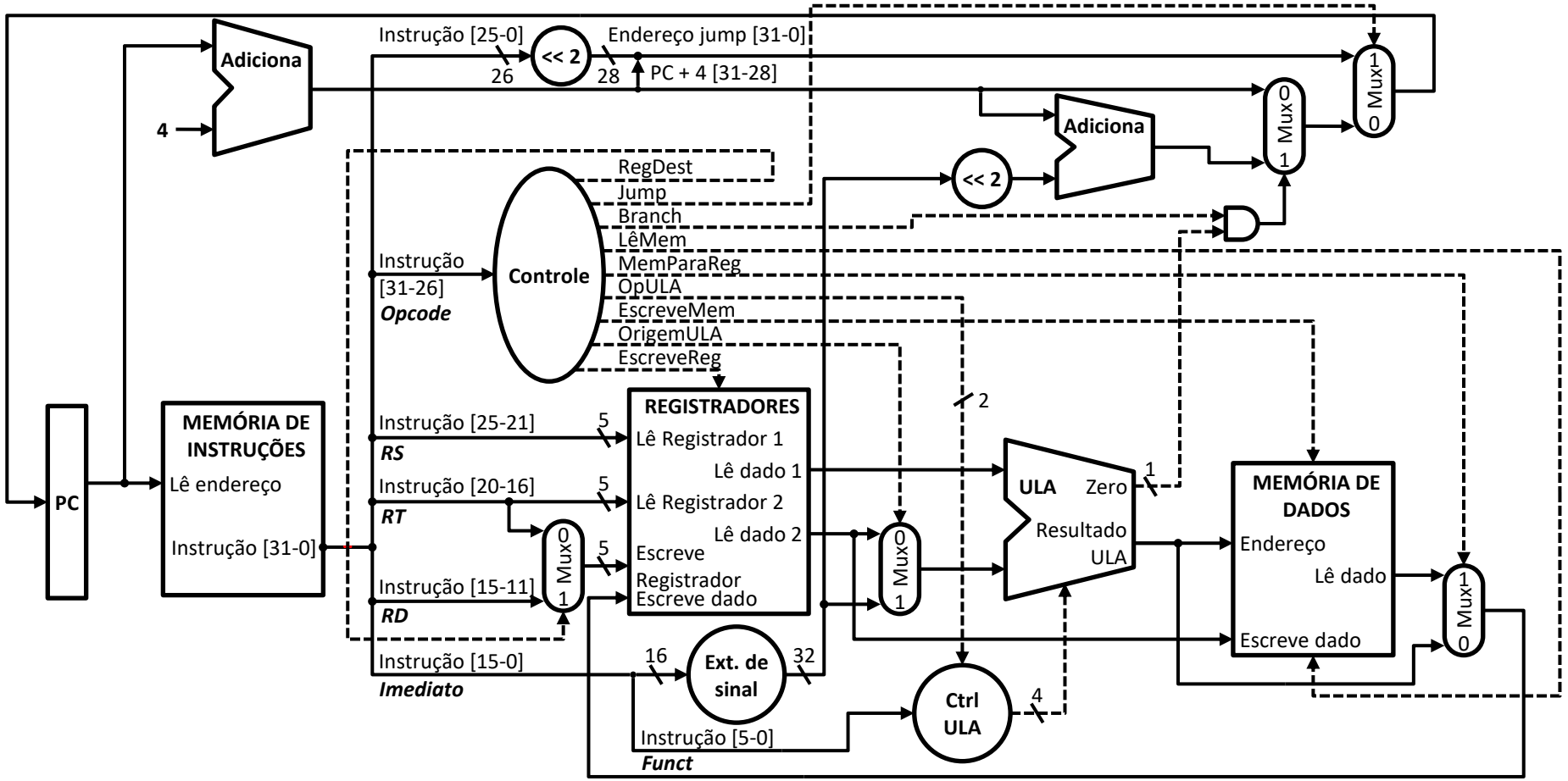
Implementação de ciclo único:



Implementação multiciclo:



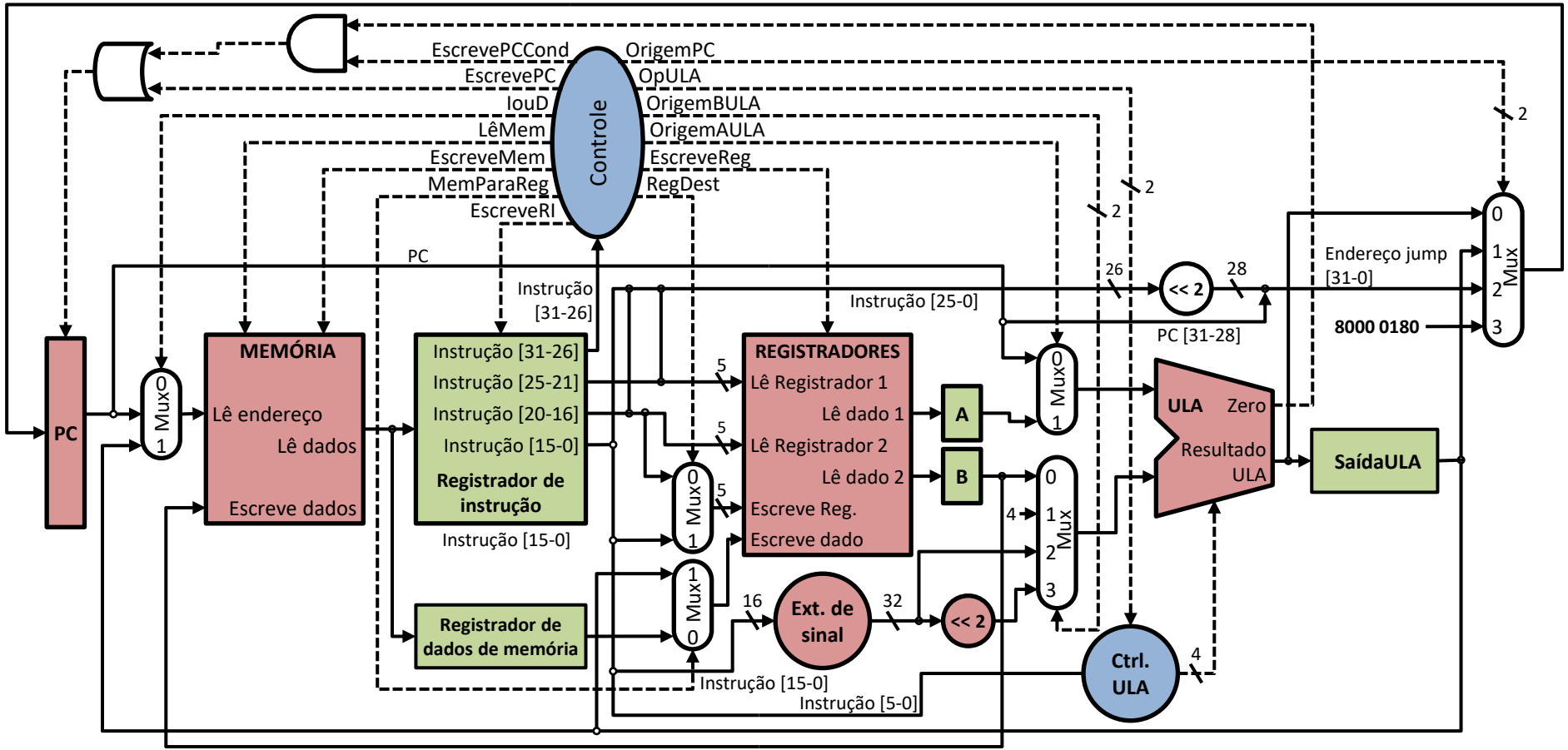
MIPS Monociclo



MIPS Multiciclo – Visão de alto nível

- Memória e ULA unificadas.
- Registradores RI e MDR
  - Os dois valores são necessários no mesmo ciclo de clock.
- Registradores A e B
  - Armazenam os valores lidos no banco de registradores
  - Mais de uma leitura é possível por execução de instrução.
- O registrador SaídaULA armazena a saída da ULA

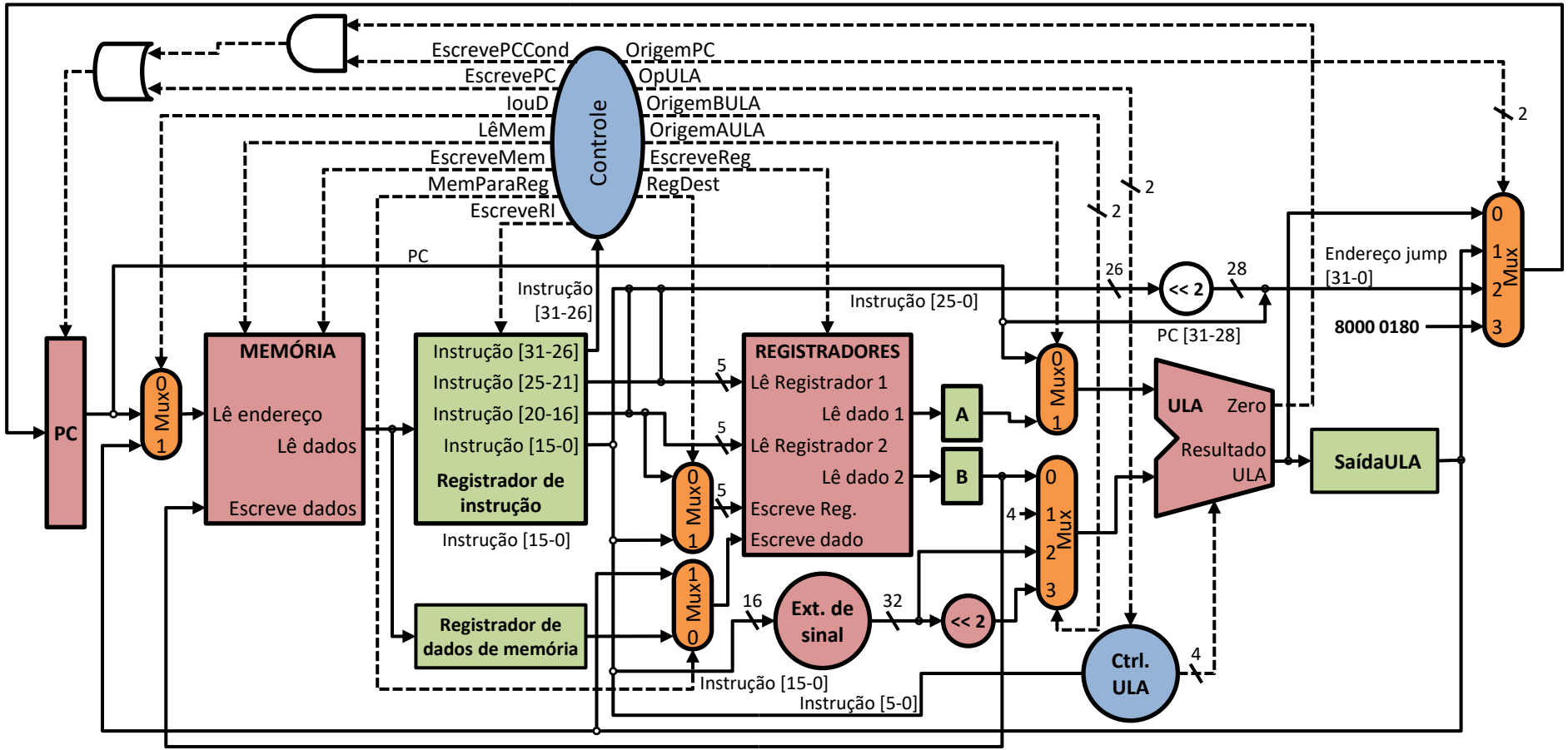
MIPS Multiciclo – Visão de alto nível



MIPS Multiciclo – Visão de alto nível

- Várias unidades funcionais são compartilhadas para diferentes finalidades
  - Multiplexadores adicionais devem ser incluídos e os existentes devem ser expandidos.
- A substituição das três ULAs do caminho de dados de ciclo único por uma única ALU
  - É necessário acomodar todas as entradas das três ULAs diferentes na implementação monociclo.
  - Um multiplexador adicional é incluído para a primeira entrada da ALU
    - Escolher entre os registradores A e o PC
- O multiplexador na segunda entrada da ULA muda de duas para quatro entradas:
  - As duas entradas adicionais são:
    - a constante 4 (incremento o PC)
    - e o campo offset com sinal estendido e deslocado (cálculo do endereço do desvio)

MIPS Multiciclo – Visão de alto nível



EXECUTANDO INSTRUÇÕES MULTICICLO

Dividindo a execução da instrução em ciclos de clock

- Implementação multiciclo exige um conjunto diferente de sinais de controle:
  - PC, memória, registradores e IR: sinal de escrita
  - Memória: sinal de leitura
  - UAL: mesmo controle da implementação monociclo
  - Multiplexador de 4 entradas: 2 linhas de controle
  - Todos os demais multiplexadores: uma linha de controle
- Para instruções jump e branch equal existem três fontes possíveis para o PC:
  - **Não tomar o branch:** PC + 4
  - **Branch:** SaidaALU (desvio condicional)
  - **Jump:** 26 LSB's do IR acrescido de 2 zeros à direita e concatenados com os 4 MSB's do PC
- Controle de escrita do PC:
  - Incremento normal e deslocamento incondicional, o PC é escrito incondicionalmente
  - Se for desvio condicional, passa o valor de SaídaULA, somente se os registradores forem iguais
  - Dois sinais de escrita: PCEsc e PCEscCond

Dividindo a execução da instrução em ciclos de clock

- Dividir as instruções em mais de um ciclo de clock:
  - Cada etapa da instrução é realizada em um ciclo de clock.
- A organização da execução de cada instrução segue um número de etapas, que depende do tipo da instrução
  - Cada etapa deve realizar apenas:
    - Uma operação de ALU;
    - Um acesso à memória ou;
    - Um acesso ao banco de registradores.
  - Com essa restrição, o ciclo de clock possui a duração de da etapa mais longa.
- Os registradores armazenam dados para os próximos ciclos da mesma instrução
  - Ex: Registradores A, B, MDR, SaídaULA.

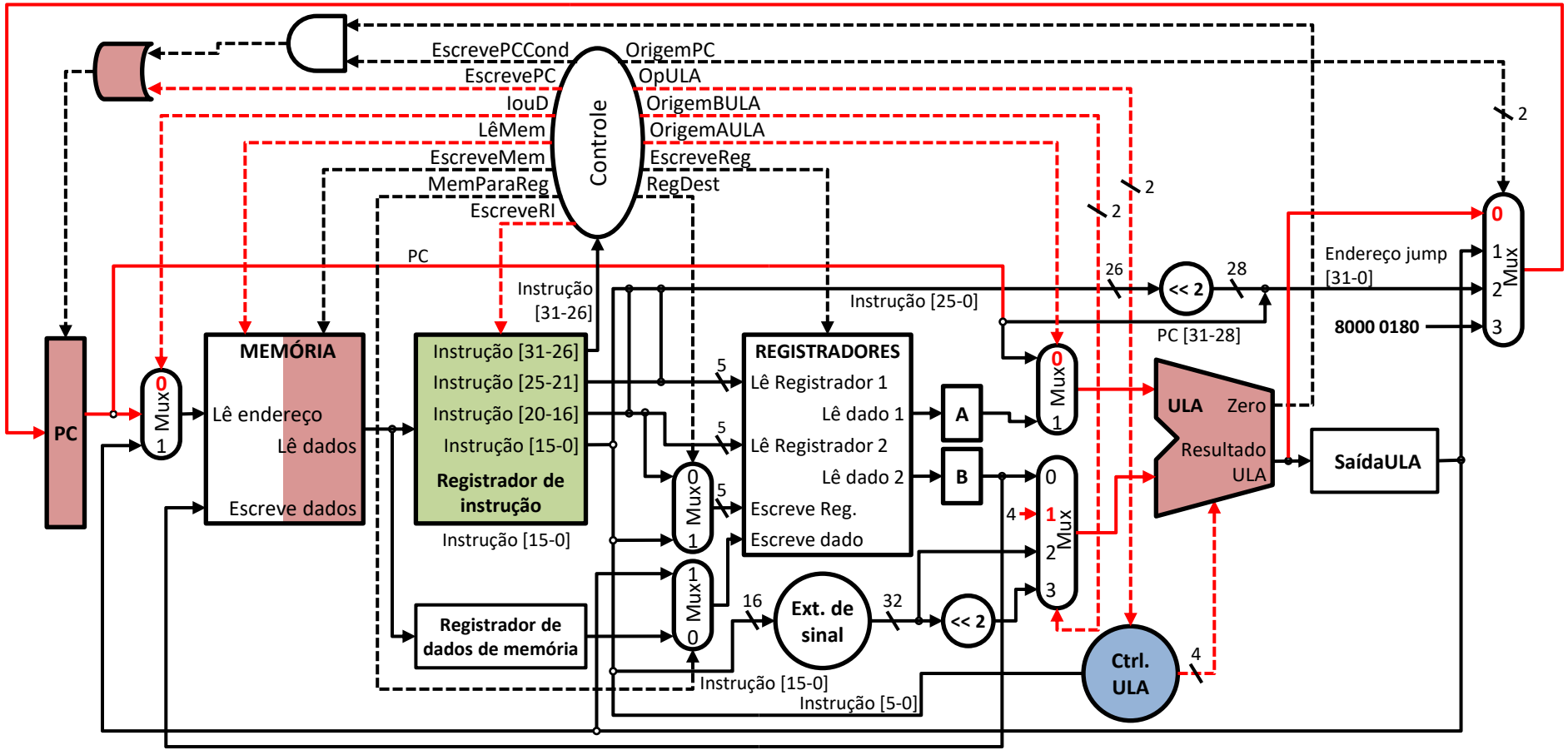
Dividindo a execução da instrução em ciclos de clock

1. **Busca da instrução**
   - (PC + Memória)
2. **Decodificação da instrução e busca dos registradores**
   - (Controle e banco de registradores)
3. **Execução, cálculo do endereço de memória ou conclusão do desvio**
   - (ULA)
4. **Acesso à memória ou conclusão de instrução tipo R**
   - (Memória ou banco de registradores)
5. **Conclusão de leitura de memória**
   - (Banco de registradores)

Dividindo a execução da instrução em ciclos de clock

1. **BUSCA DA INSTRUÇÃO**
   - IR <= Memória[PC]
   - PC <= PC+4
   - Envia o endereço armazenado no PC para a memória
     - Escreve a instrução no RI

BUSCA DA INSTRUÇÃO

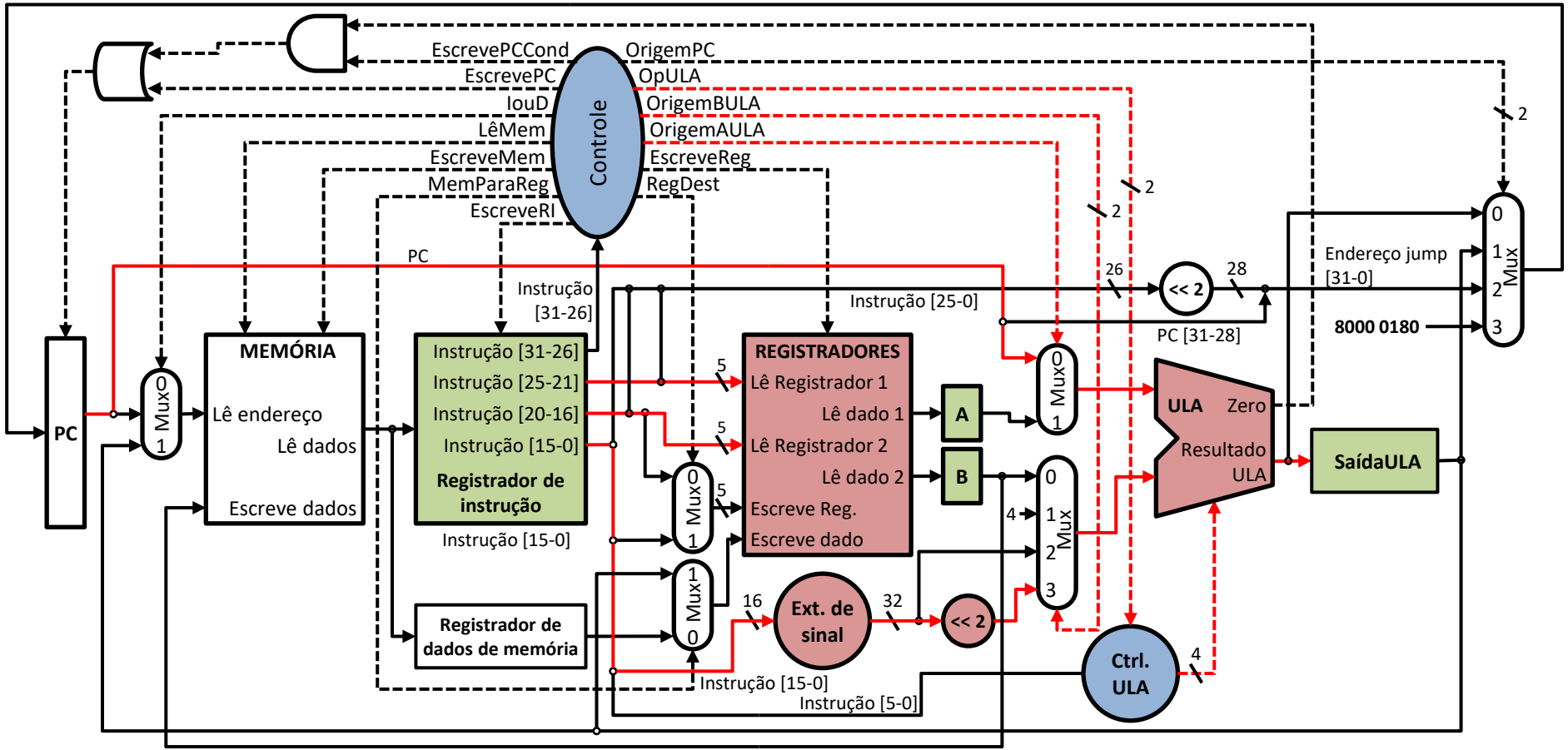


Dividindo a execução da instrução em ciclos de clock

2. DECODIFICAÇÃO E BUSCA DOS REGISTRADORES

- Não se sabe qual instrução está no IR.
- Os registradores (rs e rt) são lidos para evitar a perda de tempo durante a execução
  - Alguns valores podem ser descartadas após a decodificação da instrução
- Carrega os registradores de entrada da ALU e do endereço de desvio condicional (salvo em SaidaALU)
  - A <= Reg[IR[25-21]];
  - B <= Reg[IR[20-16]];
  - SaidaALU <= PC + extensão de sinal(IR[15-0]) << 2;

DECODIFICAÇÃO E BUSCA DOS REGISTRADORES

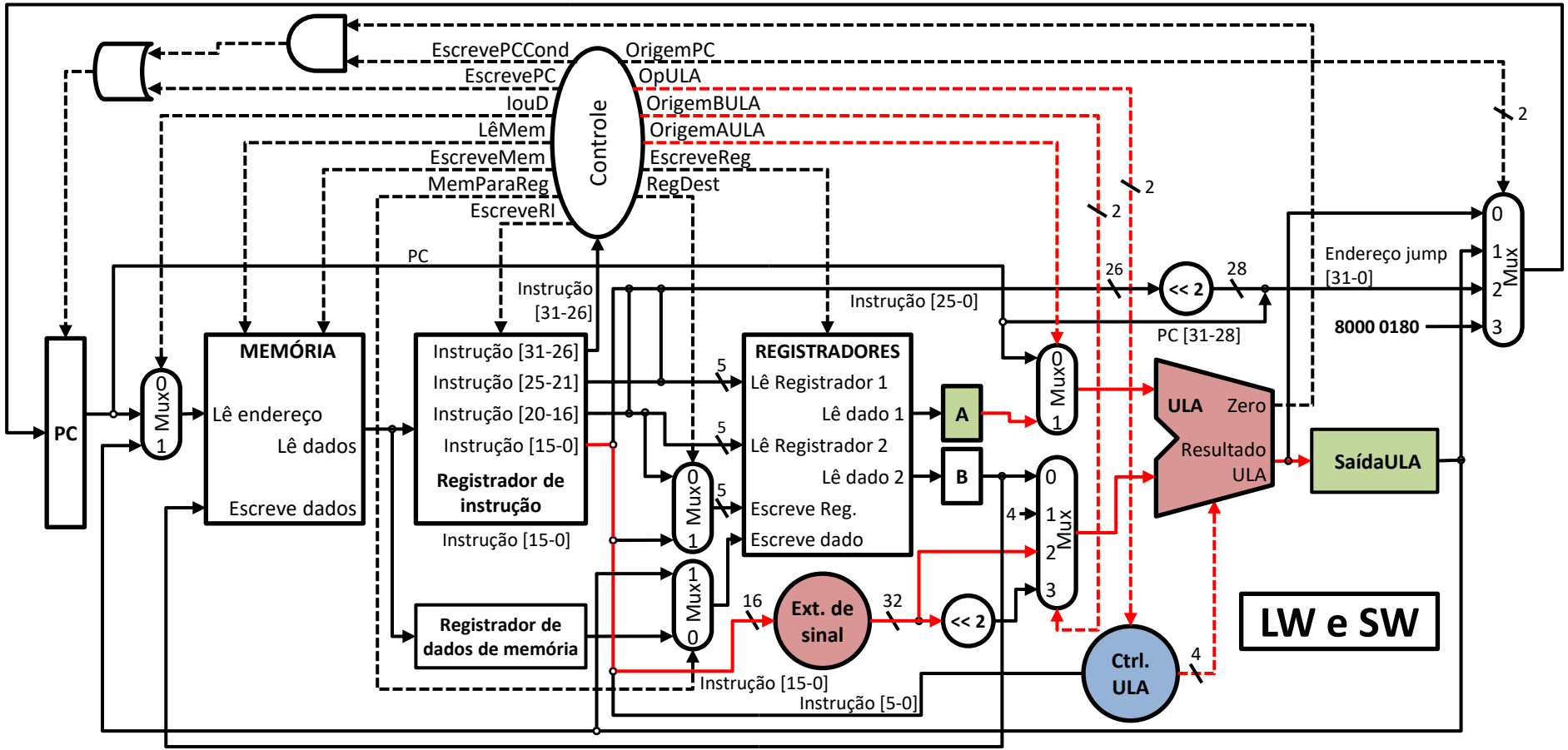


Dividindo a execução da instrução em ciclos de clock

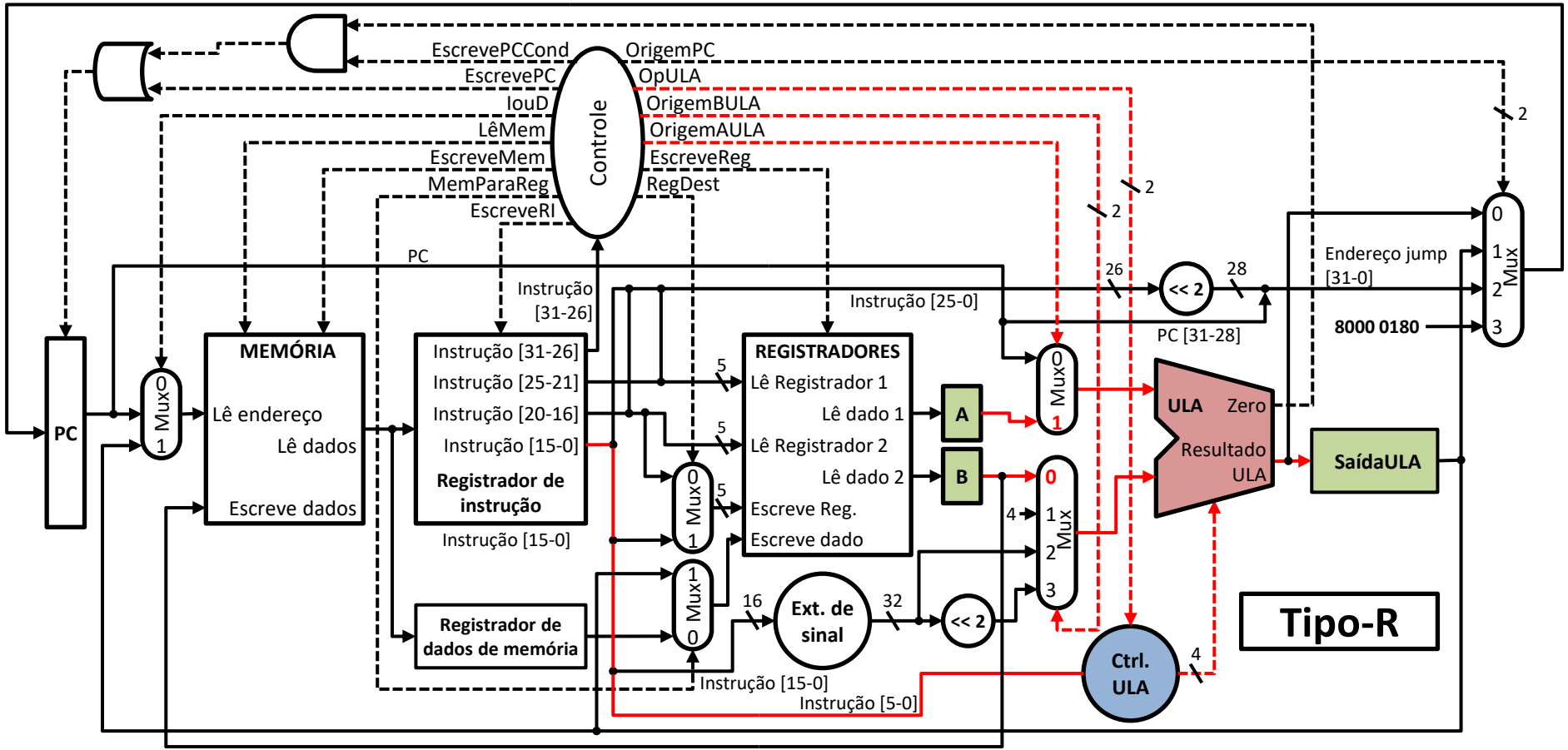
3. EXECUÇÃO, CÁLCULO DO ENDEREÇO DE MEMÓRIA OU CONCLUSÃO DO DESVIO

- a) Referência à memória (LW e SW)
  - SaidaALU <= A + extensão de sinal IR[15-0]
- b) Instrução aritmética ou lógica (tipo R)
  - SaidaALU <= A op B
- c) Desvio condicional (BEQ)
  - Se (A == B): PC <= SaidaALU
  - Finaliza a instrução
- d) Desvio incondicional (J)
  - PC <= PC [31-28] || (IR[25-0]<<2)
  - Finaliza a instrução

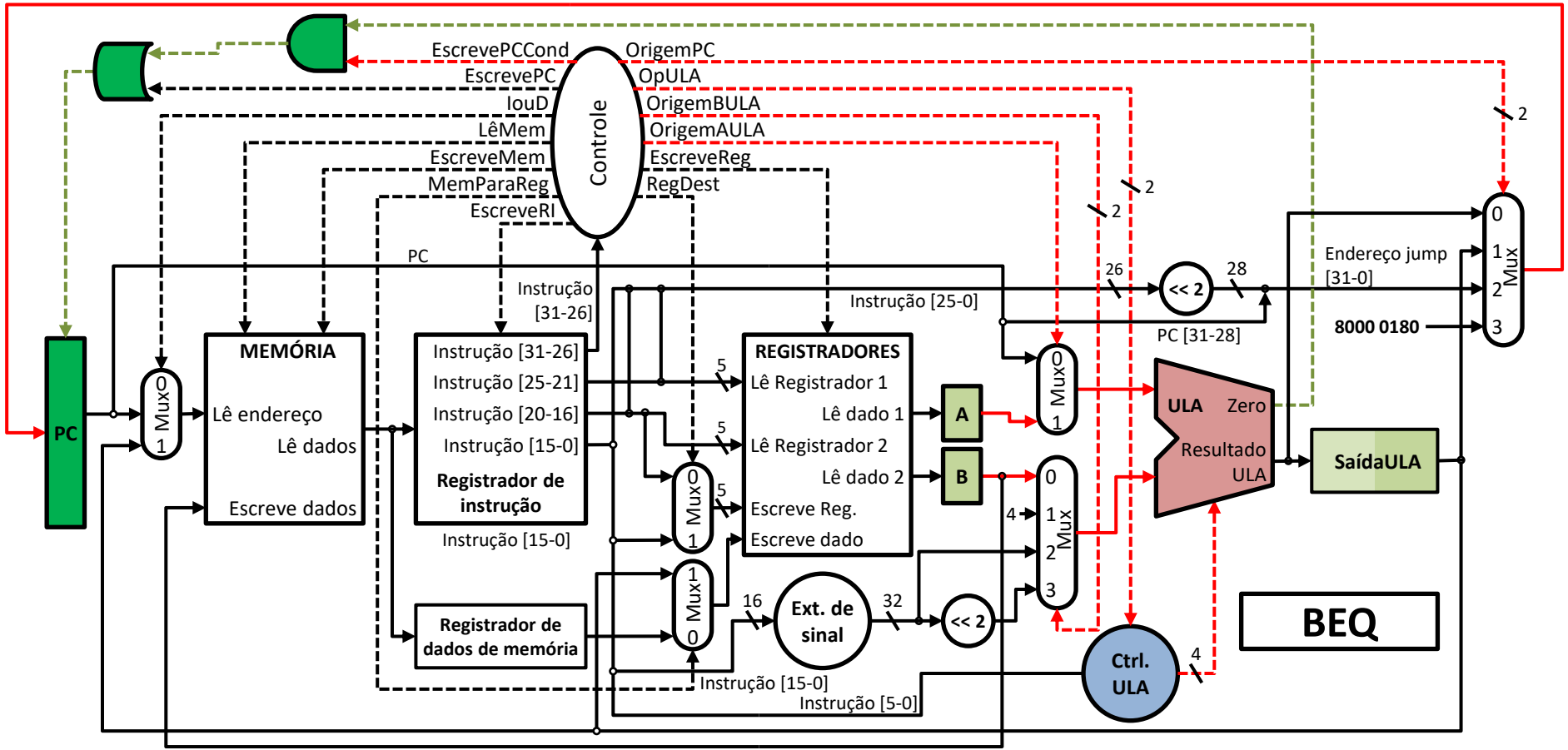
EXECUÇÃO, CÁLCULO DO ENDEREÇO DE MEMÓRIA OU CONCLUSÃO DO DESVIO



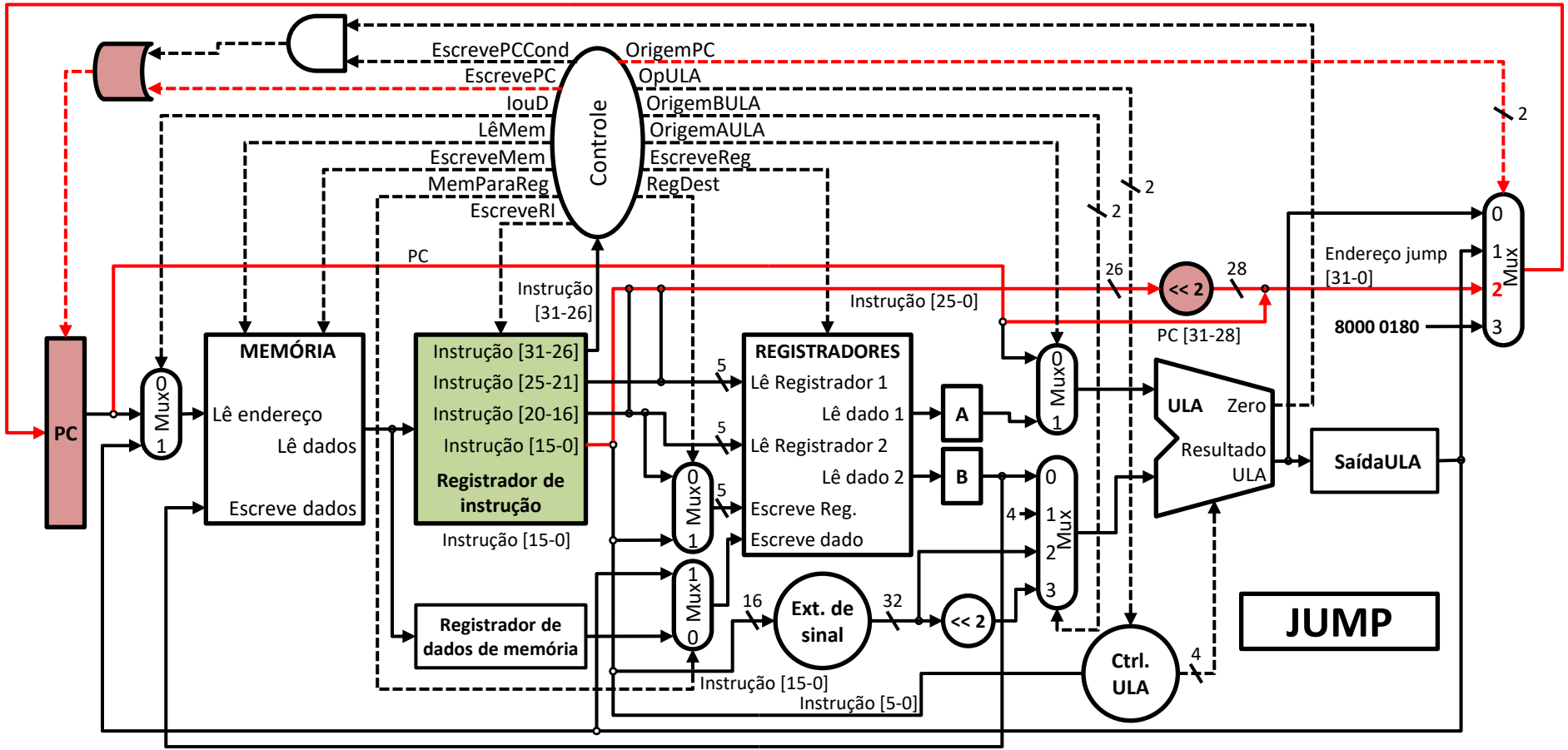
EXECUÇÃO, CÁLCULO DO ENDEREÇO DE MEMÓRIA OU CONCLUSÃO DO DESVIO



EXECUÇÃO, CÁLCULO DO ENDEREÇO DE MEMÓRIA OU CONCLUSÃO DO DESVIO



EXECUÇÃO, CÁLCULO DO ENDEREÇO DE MEMÓRIA OU CONCLUSÃO DO DESVIO



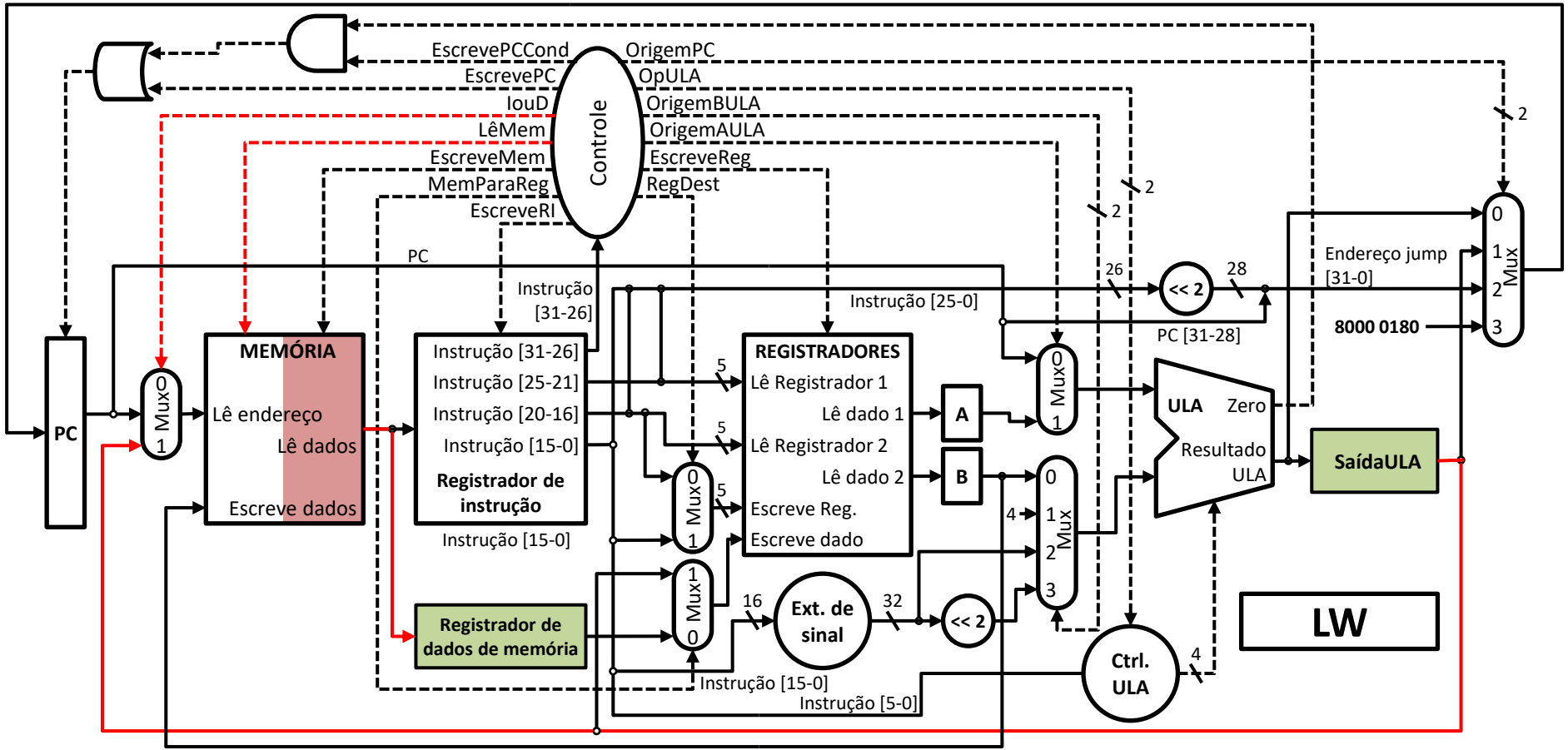
Dividindo a execução da instrução em ciclos de clock

4. ETAPA DE ACESSO À MEMÓRIA OU CONCLUSÃO DE INSTRUÇÃO TIPO R

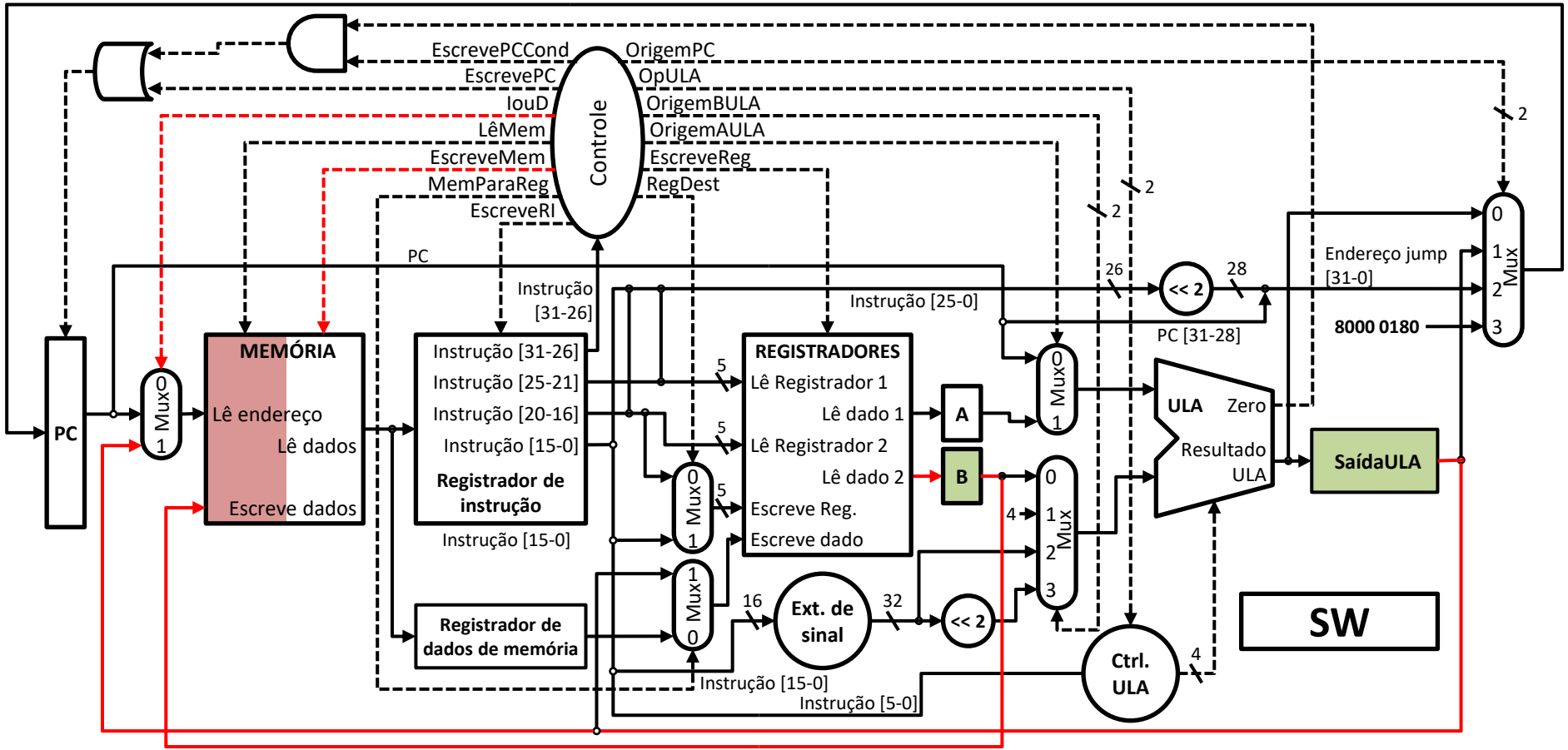
- a) Referência à memória (LW ou SW)
  - MDR = Memória[SaídaULA]; (LW)

  ou
  - Memória[SaídaULA] <= B; (SW)
    - Finaliza a instrução
- b) Instruções aritméticas ou lógicas (tipo R)
  - Reg[IR[15-11]] <= SaídaULA;
    - Finaliza a instrução

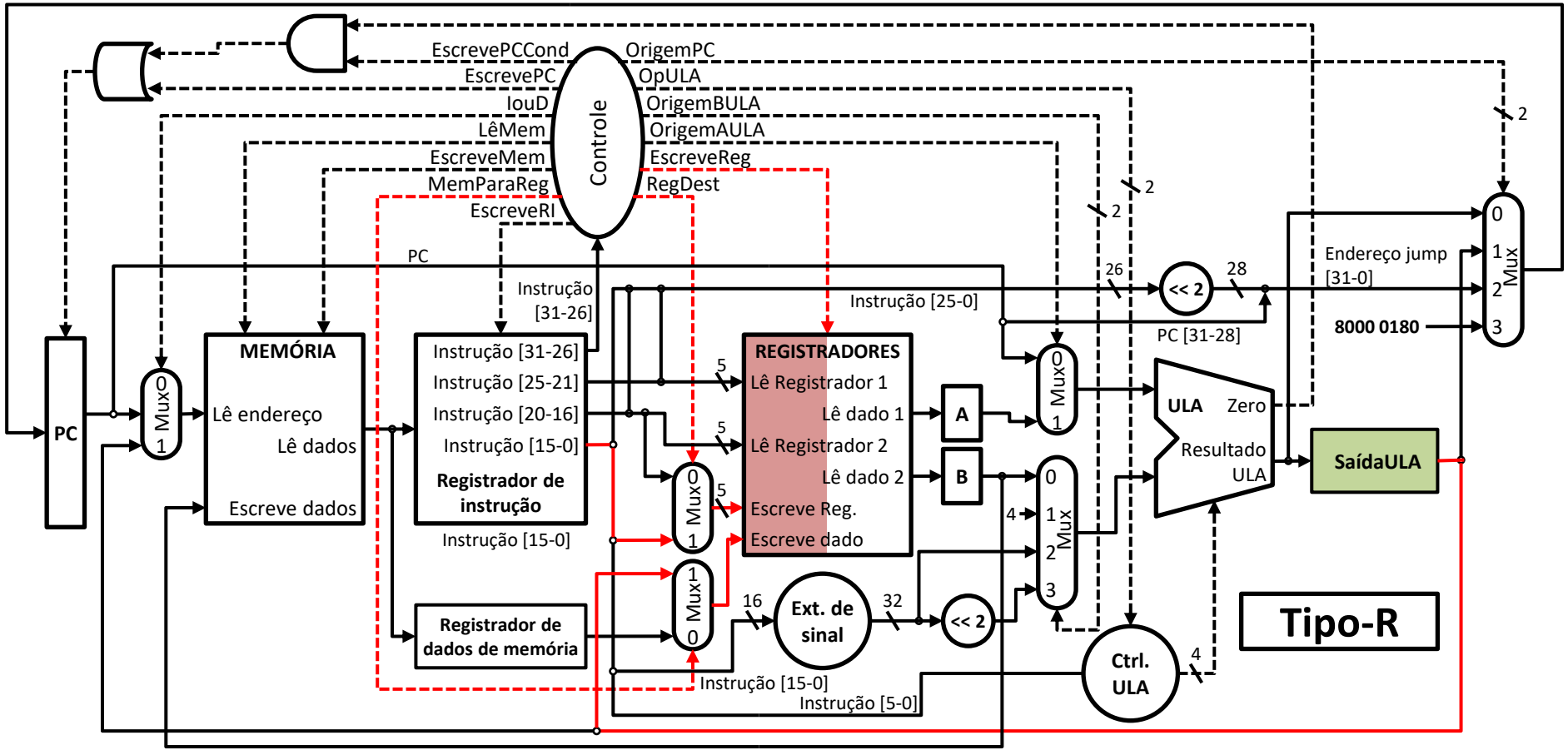
ETAPA DE ACESSO À MEMÓRIA OU CONCLUSÃO DE INSTRUÇÃO TIPO R



ETAPA DE ACESSO À MEMÓRIA OU CONCLUSÃO DE INSTRUÇÃO TIPO R



ETAPA DE ACESSO À MEMÓRIA OU CONCLUSÃO DE INSTRUÇÃO TIPO R

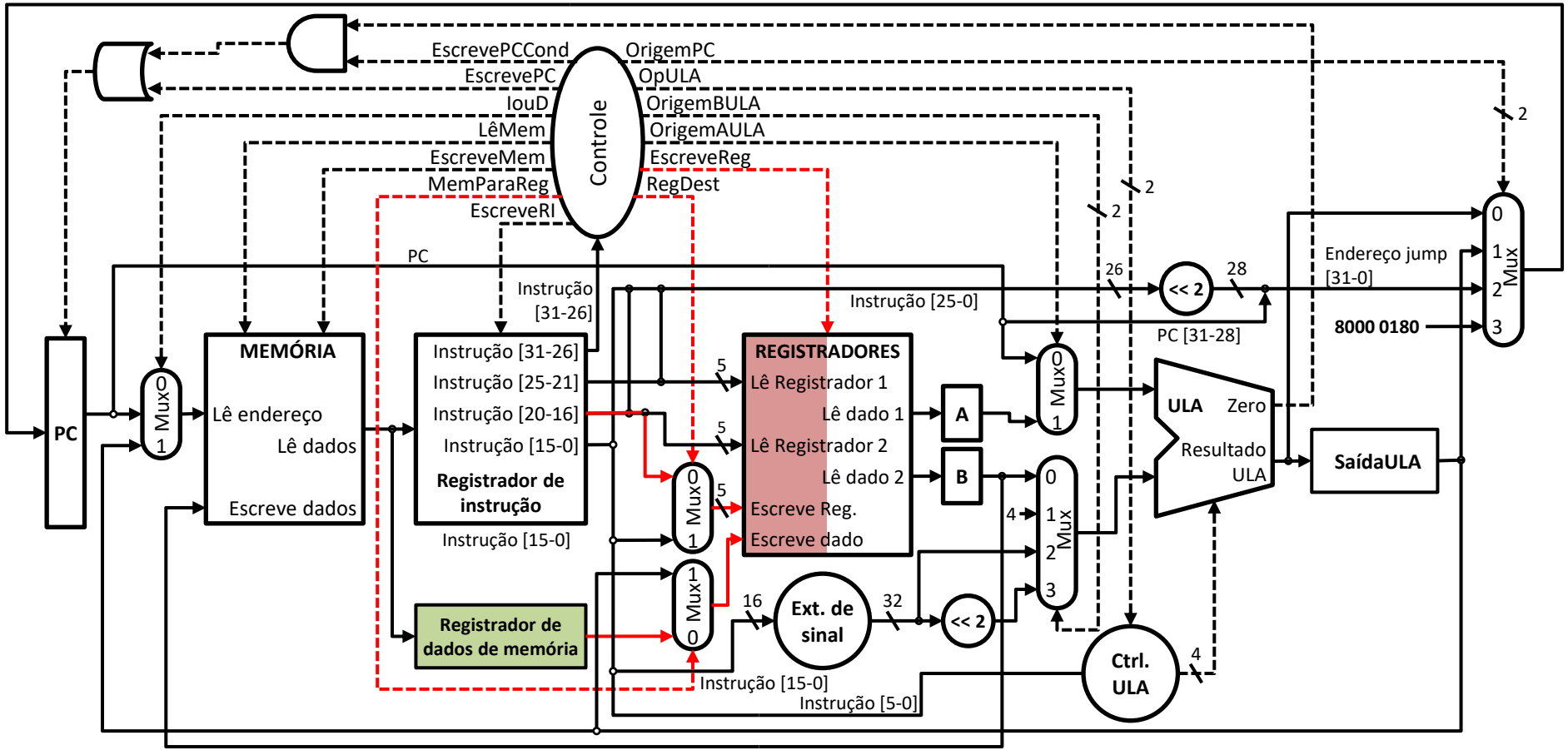


Dividindo a execução da instrução em ciclos de clock

5. ETAPA DE CONCLUSÃO DE ACESSO À MEMÓRIA (LW)

- Reg[IR[20-16]] <= MDR

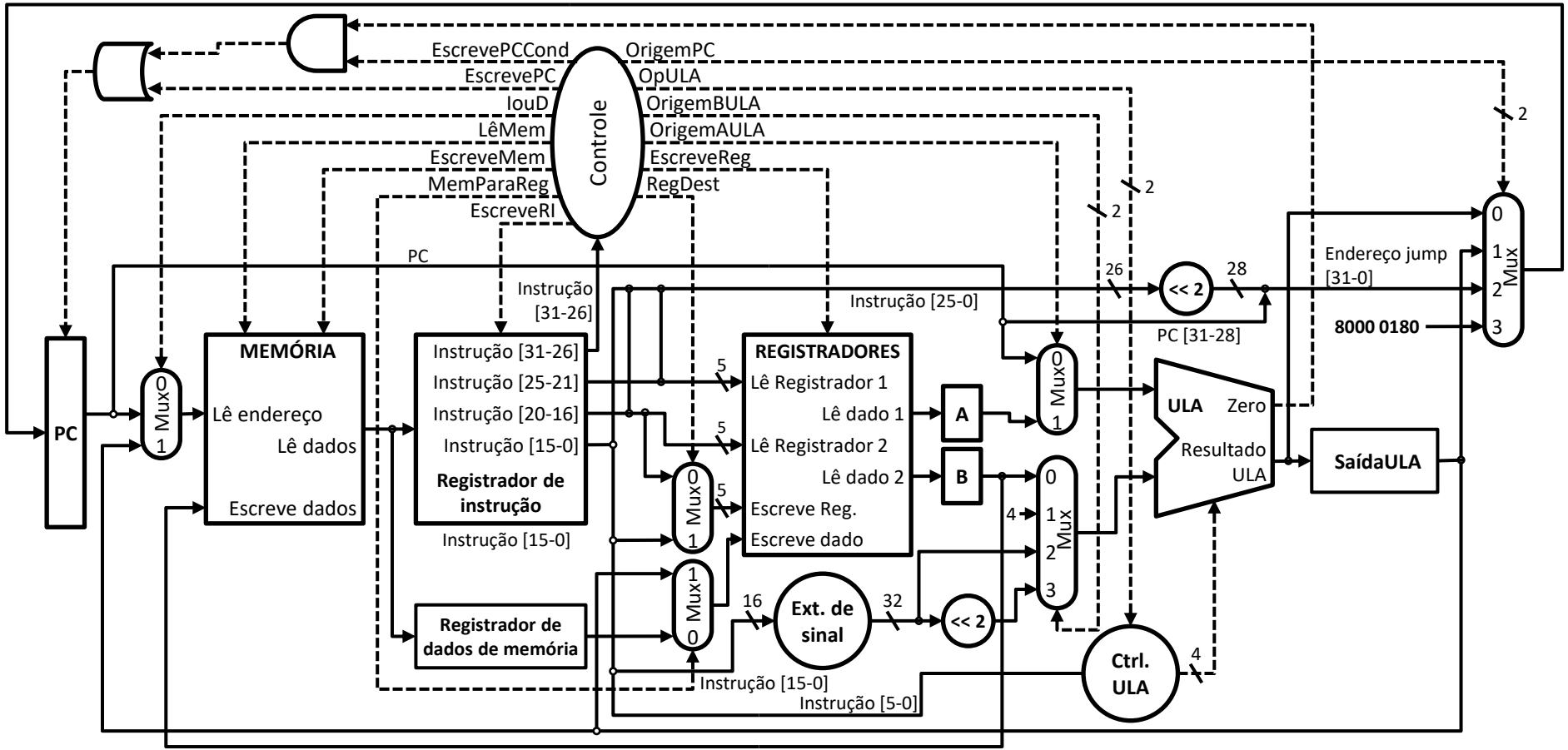
ETAPA DE CONCLUSÃO DE ACESSO À MEMÓRIA (LW)



Dividindo a execução da instrução em ciclos de clock

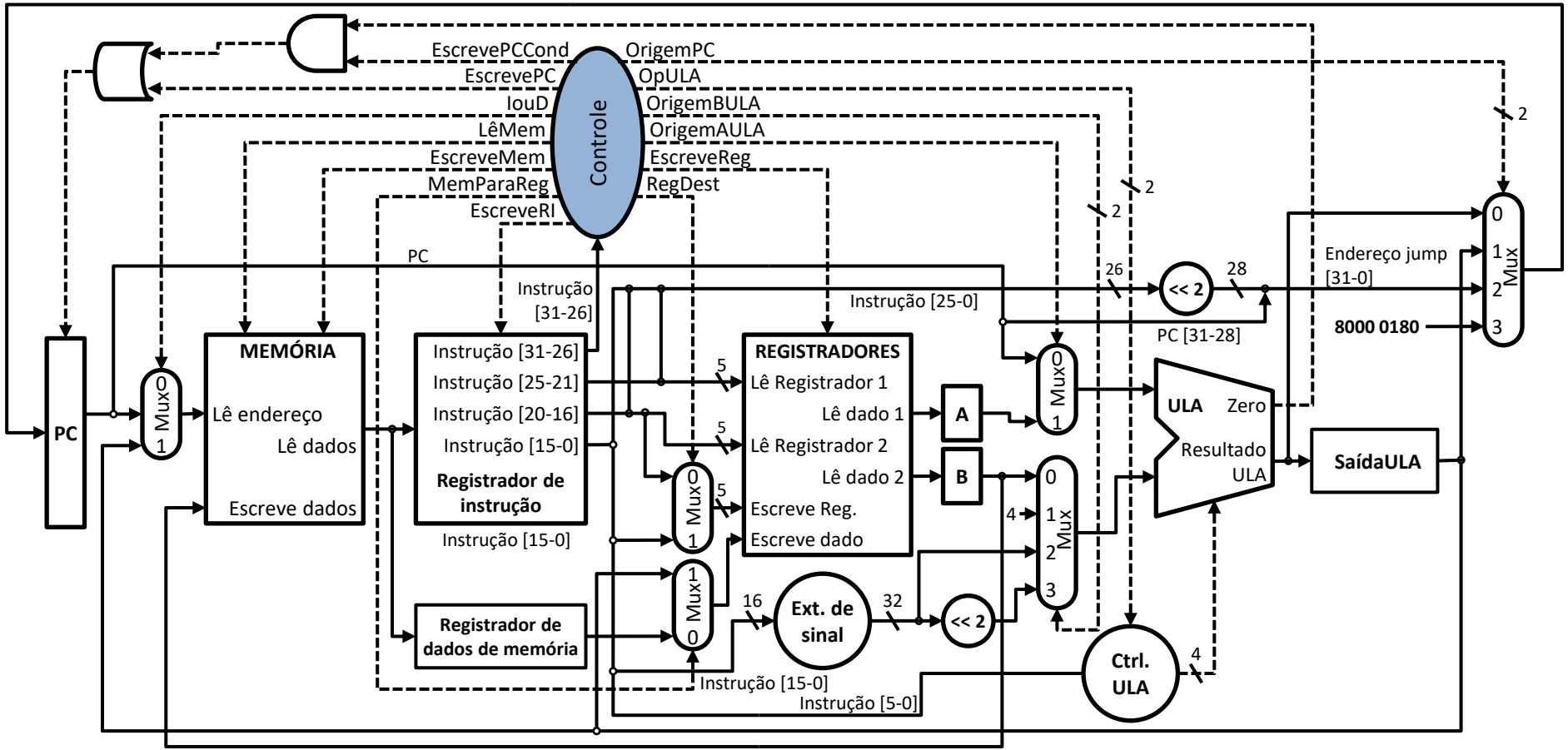
Etapa	Ação para instruções tipo R	Ação para instruções de acesso à memória	Ação para desvios	Ação para jumps
Busca da instrução	IR <= Memória[PC] PC <= PC + 4			
Decodificação da instrução e busca dos registradores	A <= Reg[IR[25:21]] B <= Reg[IR[20:16]] SaídaULA <= PC + (estende-sinal (IR[15:0]) << 2)			
Execução, cálculo do endereço ou conclusão do desvio/jump	SaídaALU <= A op B	SaídaALU <= A + estende_sinal (IR[15:0])	If(A==B) PC <= SaídaALU	PC <= {PC[31:28], (IR[25:0], 2'b00)}
Acesso à memória ou conclusão de instrução do tipo R	Reg[IR[15:11]] <= SaídaALU	Load: MDR <= Memória[SaídaALU] ou Store: Memória[SaídaALU] <= B		
Conclusão da leitura da memória		Load: Reg[IR[20:16]] <= MDR		

Dividindo a execução da instrução em ciclos de clock



DEFININDO O CONTROLE

Definindo o controle



Definindo o controle

CC		EscrevePC Cond	EscrevePC	IouD	LêMem	Escreve Mem	MemPara Reg	EscreveRI	OrigemPC	OpULA	OrigemB ULA	OrigemA ULA	Escreve Reg	RegDest
1		0	1	0	1	0	X	1	00 (0)	00	01 (1)	0	0	X
2		0	0	X	0	0	X	0	XX	00	11 (3)	0	0	X
3	L/A	0	0	X	0	0	X	0	XX	10	00	1	0	X
	LW	0	0	X	0	0	X	0	XX	00	10	1	0	X
	SW	0	0	X	0	0	X	0	XX	00	10	1	0	X
	BEQ	1	0	X	0	0	X	0	01 (1)	01	00	1	0	X
	J	0	1	X	0	0	X	0	10 (2)	XX	XX	X	0	X
4	L/A	0	0	X	0	0	1	0	XX	XX	XX	X	1	1
	LW	0	0	1	1	0	X	0	XX	XX	XX	X	0	X
	SW	0	0	1	0	1	X	0	XX	XX	XX	X	0	X
5	LW	0	0	X	0	0	0	0	XX	XX	XX	X	1	0

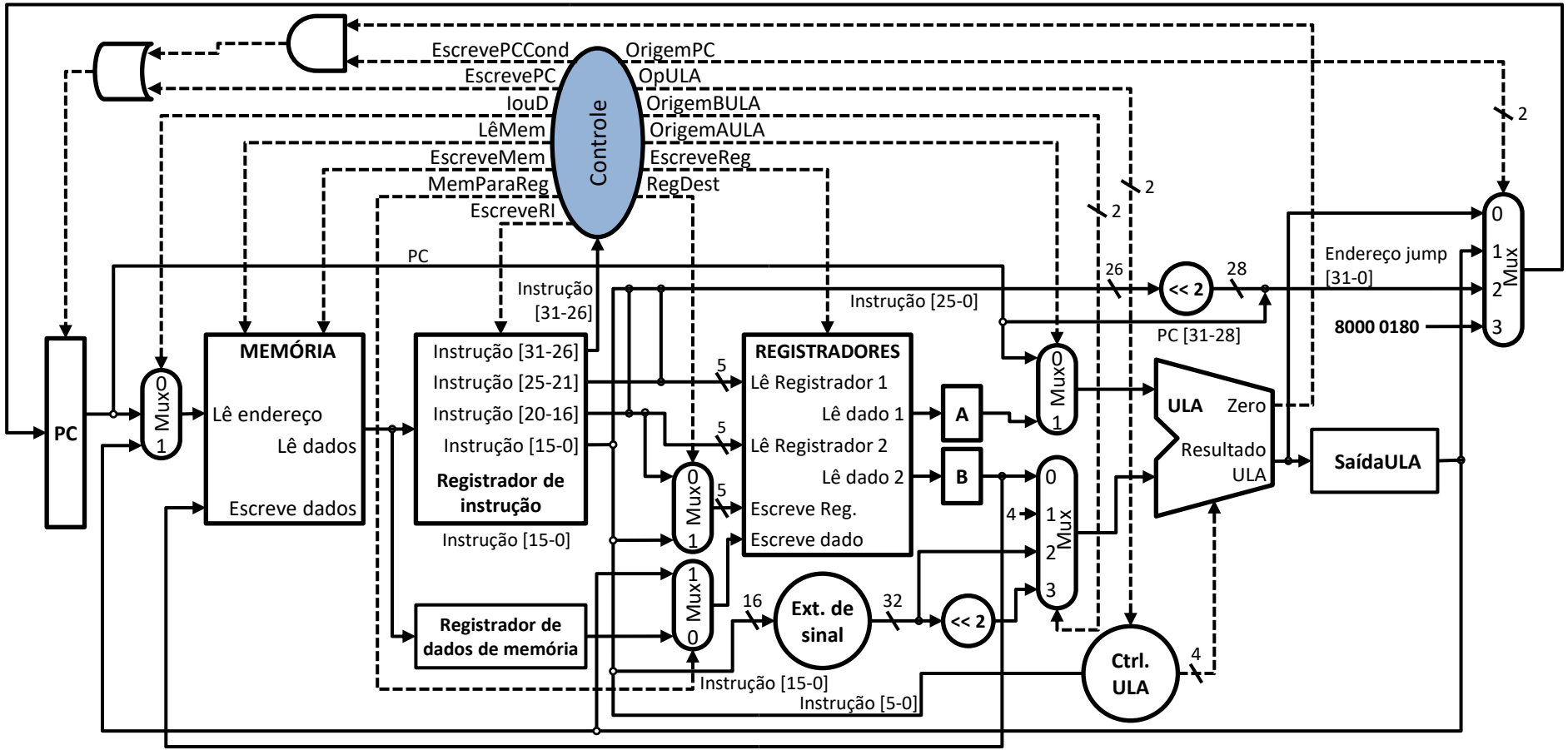
Sinais de controle de 1 bit

Nome do sinal	Efeito quando inativo	Efeito quando ativo
RegDst	O número do registrador de destino do banco de registradores para a entrada “Registrador para escrita” vem do campo rt.	O número do registrador de destino do banco de registradores para a entrada “Registrador para escrita” vem do campo rd.
EscreveReg	Nenhum.	O registrador de uso geral selecionado pelo número na entrada “Registrador para escrita” é escrito como valor da entrada “Dados para escrita”.
OrigAALU	O primeiro operando da ALU é o PC.	O primeiro operando da ALU vem do registrador A.
LeMem	Nenhum.	O conteúdo da memória no local especificado pela entrada Endereço é colocado na saída Dados da memória.
EscreveMem	Nenhum.	O conteúdo da memória no local especificado pela entrada Endereço é substituido pelo valor na entrada “Dados para escrita”.
MemparaReg	O valor enviado para a entrada “Dados para escrita” do banco de registradores vem de SaídaALU.	O valor enviado para a entrada “Dados para escrita” do banco de registradores vem do MDR.
IouD	O PC é usado para fornecer o endereço para a unidade de memória.	SaídaALU é usado para fornecer o endereço para a unidade de memória.
IRWrite	Nenhum.	A saída da memória é escrita no IR.
EscrevePC	Nenhum.	O PC é escrito; a origem é controlada por OrigPC.
EscrevePCCond	Nenhum.	O PC é escrito se a saída Zero da ALU também estiver ativa.

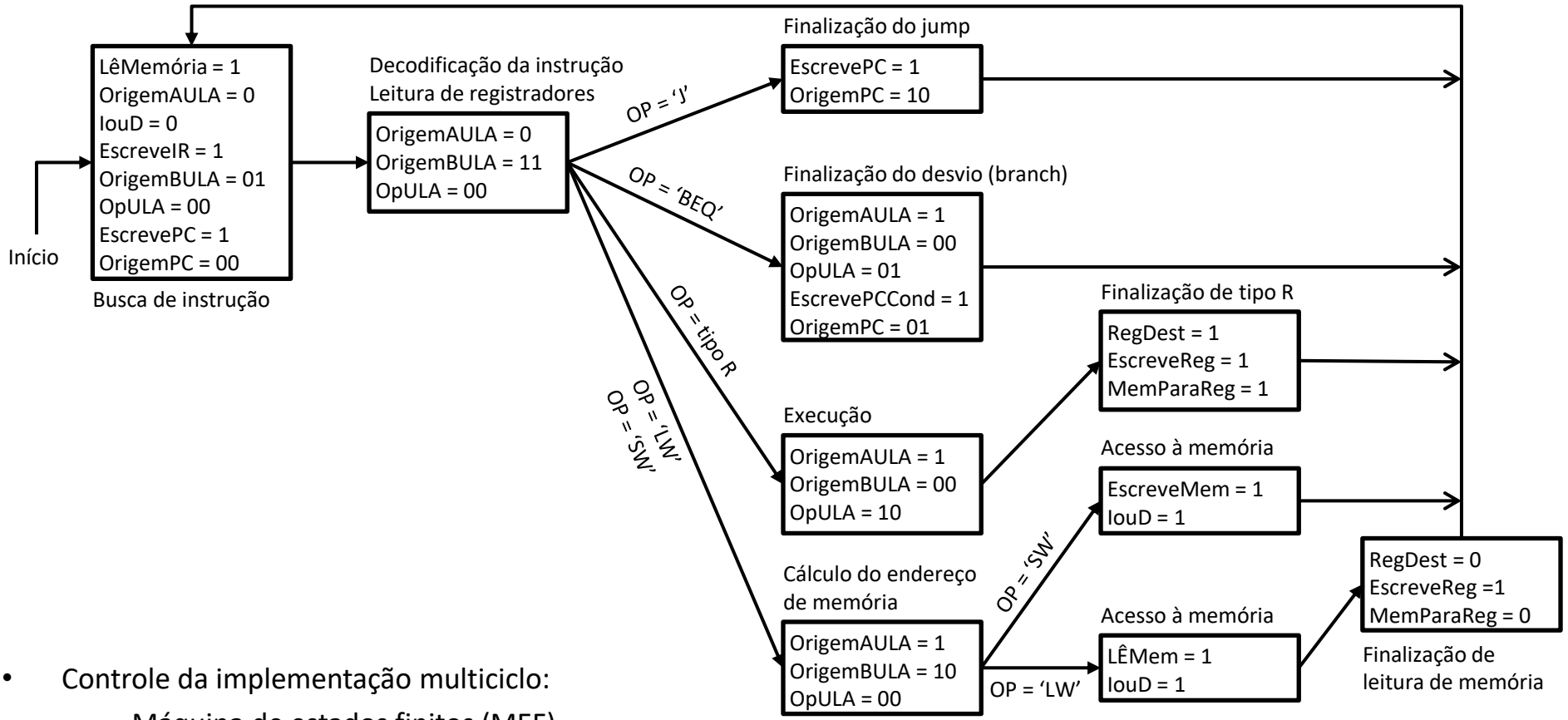
Sinais de controle de 2 bit

Nome do sinal	Valor (binário)	Efeito
OpALU	00	A ALU realiza uma operação de adição.
	01	A ALU realiza uma operação de subtração.
	10	O campo funct da instrução determina a operação da ALU.
OrigBALU	00	A segunda entrada para a ALU vem do registrador B.
	01	A segunda entrada da ALU é a constante 4.
	10	A segunda entrada da ALU são os 16 bits menos significativos com sinal estendido do IR.
	11	A segunda entrada da ALU são os 16 bits menos significativos com sinal estendido do IR deslocados em 2 bits para a esquerda.
OrigPC	00	A saída da ALU (PC + 4) é enviada ao PC para escrita.
	01	O conteúdo da SaídaALU (o endereço de destino do desvio) é enviado ao PC para escrita.
	10	O endereço de destino do jump (IR[25:0]) deslocado de 2 bits para a esquerda e concatenado com PC + 4[31:28] é enviado ao PC para escrita.

Definindo o controle



Definindo o controle



- Controle da implementação multiciclo:
  - Máquina de estados finitos (MEF)

Bibliografia

- PATTERSON, D. A.; HENNESSY, J. L. **Organização e Arquitetura de computadores: a interface hardware/software**. 3. ed. Rio de Janeiro: Elsevier, 2005.
  - CAPÍTULO 5
- Notas de aula da Profa. Mary Jane Irwin
  - CSE 331 Computer Organization and Design
  - http://www.cse.psu.edu/research/mdl/mji/mjicourses

FIM